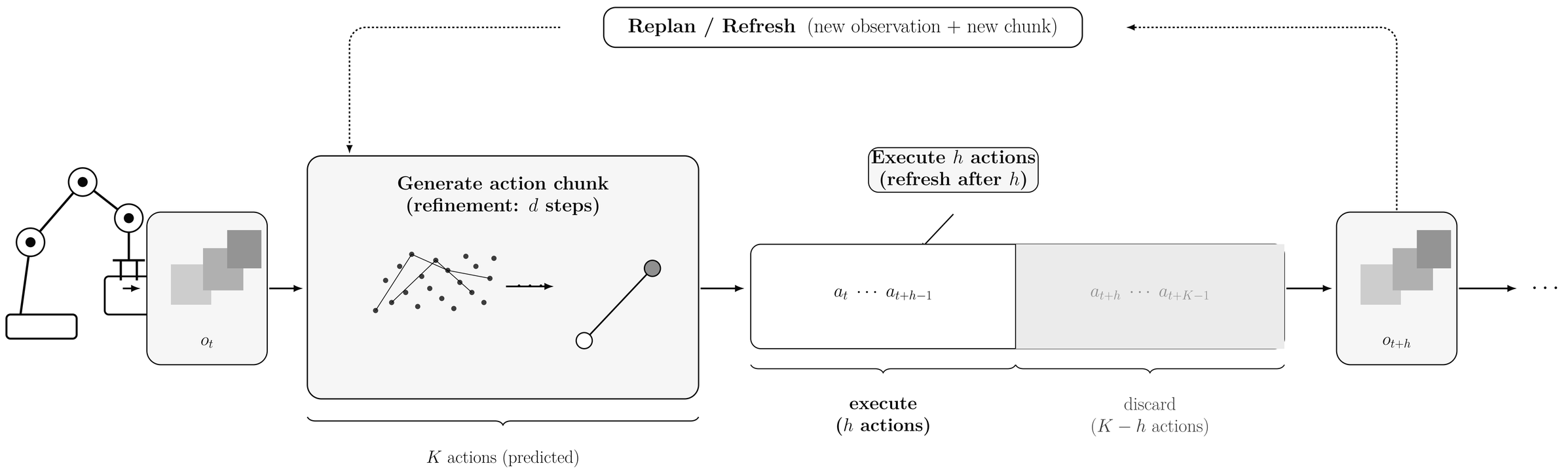
Refinement vs. Refresh

Compute Allocation for Closed-Loop Robot Policies

Alba María Téllez Fernández, MAAT Institute (alba.tellez@maatinstitute.com)

Chunked Closed-Loop Policy



K: **chunk length**
(total predicted actions)

d: **refinement**
(denoising steps)

h: **refresh**
(actions before replanning)

Setup (held-out)

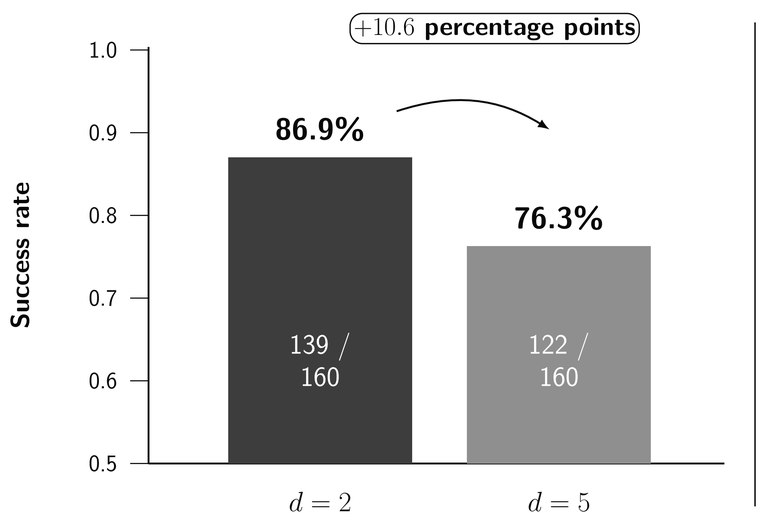
SmolVLA (frozen) $K = 50,\ d \in \{2, 5\}$

LIBERO (simulation) $h \in \{2, 5\}$ 8 tasks, 1,120 attempts

?

Under a fixed inference budget, should we allocate compute to additional denoising steps (**refinement**) or to more frequent observation-conditioned replanning (**refresh**)?

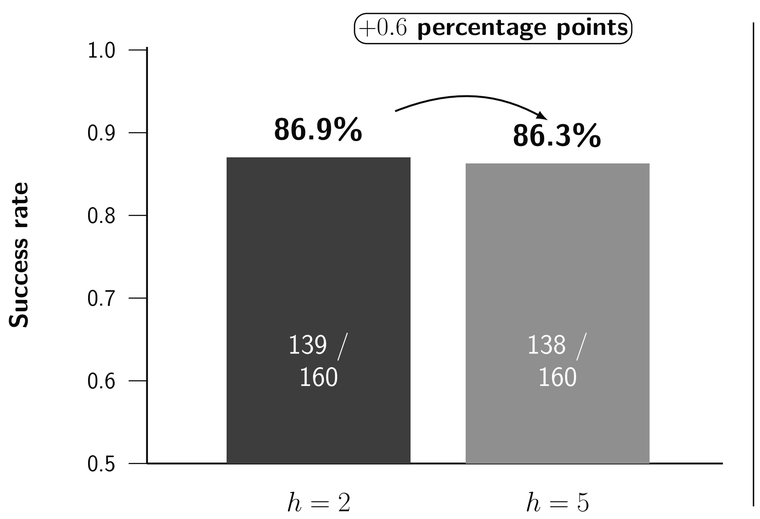
Result 1: Fewer Denoising Steps Improve Success



Takeaway

At $h = 2$, reducing denoising from 5 to 2 steps raises success by **10.6 pp** and uses **5.9% less** arithmetic per control step.

Result 2: Less Frequent Refresh Is Much Cheaper



Takeaway

At $d = 2$, $h = 5$ gives 138/160 successes versus 139/160 for $h = 2$, with **60.7% less** arithmetic per attempt. Equivalent quality is not established.

Why This Matters

Closed-loop inference has two levers: refine the current chunk or refresh from a new observation. The primary comparison favors less refinement, while the prefix comparison identifies a much cheaper candidate. Together, they turn compute allocation into a control decision.

Interpretation & Limitations

Takeaways

- Primary: less refinement improved success at fixed $h = 2$.
- Secondary: $h = 5$ is much cheaper at $d = 2$, but remains a descriptive candidate.

Limitations

- One frozen SmolVLA checkpoint in synchronous LIBERO.
- Changing h changes refresh timing and action positions together.
- Similar success is not an equivalence claim; arithmetic is not latency.